

TECHNICAL REPORT

Electro-Thermal PINN

A Mesh-Free Solver for Coupled Maxwell's Equations
and Heat Transfer

Mohammad Hussein Ghafoori

Backend & AI Engineer | Physics-Inspired Problem Solver

✉ king.mohamd.09876@gmail.com



[mohammad-hussein-dev](#)



[mohammad-hussein-dev](#)



[mohammad-hussein-dev.github.io](#)



[mohammad-hussein-dev](#)

🔗 Repo:



[electro-thermal-pinn](#)



[electro-thermal-pinn](#)

July 31, 2026

Version 1.0 | Status: Production-Ready

PINN | Physics-Informed Neural Networks | Maxwell's Equations | Heat Transfer | Joule Heating | Deep
Learning

Executive Summary

This report presents a comprehensive **Physics-Informed Neural Network (PINN)** framework for solving coupled electromagnetic-thermal systems without mesh generation.

Key Achievement	Details
Mesh-Free Simulation	Eliminates FEM/FVM mesh generation, reducing preprocessing from hours to minutes
Exceptional Accuracy	Relative L2 errors below 0.1% for electric field with only 500 collocation points
Three Architectures	MLP (fast), MLPPINN (balanced), TransformerPINN (highest accuracy)
Production-Ready	Modular Python implementation with PyTorch, interactive menu, Streamlit dashboard

The framework predicts electric field (**E**), magnetic field (**H**), and temperature (*T*) simultaneously, with training time of approximately **5–12 minutes on CPU**.

This work establishes PINNs as a viable, mesh-free alternative for multi-physics simulations with significant potential for industrial applications in electronic packaging, battery design, energy systems, and biomedical engineering.

Contents

1	Abstract	1
2	Introduction	1
3	Problem Formulation	2
3.1	Governing Equations	2
3.2	Boundary and Initial Conditions	2
3.3	Non-dimensionalization	2
4	PINN Architecture	3
4.1	Neural Network Approximation	3
4.2	Architectural Variants	3
4.3	Automatic Differentiation	3
5	Loss Function and Training	4
5.1	Total Loss Function	4
5.2	Training Procedure	4
5.3	Hyperparameters	4
6	Results and Discussion	5
6.1	Validation with MLP Architecture	5
6.2	Architectural Comparison	5
6.3	Error Analysis	5
6.4	Visualization of Results	5
7	Industrial Applications	5
8	Conclusion	6
9	Future Work	7
10	References	7
A	Code Repository	8
A.1	Project Structure	8
B	Environment Setup	8
B.1	Installation	8
B.2	Dependencies	9
C	Usage Examples	9
C.1	Example 1: Running the Interactive Terminal Interface	9
C.2	Example 2: Training a Model Programmatically	9
C.3	Example 3: Running the Streamlit Dashboard	10
D	License	10

1 Abstract

This report presents a comprehensive Physics-Informed Neural Network (PINN) framework for solving coupled electromagnetic-thermal systems without the need for mesh generation. The governing equations, including Maxwell's equations and the heat equation with Joule heating, are embedded directly into the neural network's loss function. Three distinct architectures are implemented and compared: a standard MLP, an enhanced MLPPINN with Fourier features and SwiGLU, and a TransformerPINN with RoPE and GQA. The framework predicts electric field ($\mathbf{E}$), magnetic field ($\mathbf{H}$), and temperature (T) simultaneously using PyTorch's automatic differentiation. Results demonstrate exceptional accuracy, with relative L2 errors as low as 9.09×10^{-4} for the electric field using only 500 collocation points and approximately 5 minutes of training on CPU. This work establishes PINNs as a viable, mesh-free alternative for multi-physics simulations with significant potential for industrial applications in electronic packaging, battery design, and energy systems.

2 Introduction

Numerical simulation of multi-physics systems, particularly those involving coupled electromagnetic and thermal phenomena, is essential in modern engineering design. Applications range from electronic packaging and thermal management of integrated circuits to battery design and induction heating systems. Traditional approaches rely on mesh-based methods such as the Finite Element Method (FEM) or Finite Volume Method (FVM), which, while powerful, present significant challenges:

- **Mesh Generation:** Complex geometries require time-consuming and expertise-driven mesh construction.
- **Computational Cost:** High-fidelity simulations demand substantial computational resources and time.
- **Multi-Physics Coupling:** Handling interactions between different physical domains often requires complex solvers and careful numerical treatment.

Physics-Informed Neural Networks (PINNs) have emerged as a transformative alternative. First introduced by Raissi et al. (2019), PINNs embed the governing partial differential equations (PDEs) directly into the neural network's loss function, enabling the approximation of solutions without labeled data. Automatic differentiation replaces traditional numerical differentiation, eliminating the need for mesh discretization.

This work presents a comprehensive PINN framework for solving coupled Maxwell's equations and the heat equation with Joule heating. The contributions are:

1. A mesh-free, fully-differentiable PINN that predicts electric field ($\mathbf{E}$), magnetic field ($\mathbf{H}$), and temperature (T) simultaneously.
2. Implementation and comparison of three neural architectures: MLP, MLPPINN, and TransformerPINN.
3. Validation with relative L2 errors below 0.1% using minimal computational resources.
4. A modular, production-ready codebase with interactive model selection.

3 Problem Formulation

3.1 Governing Equations

We consider the coupled electromagnetic-thermal problem in a one-dimensional domain with Joule heating. The system is governed by:

Maxwell's Equations (1D):

$$\frac{\partial E}{\partial t} + \frac{1}{\epsilon} \frac{\partial H}{\partial x} = 0 \quad (1)$$

$$\frac{\partial H}{\partial t} + \frac{1}{\mu} \frac{\partial E}{\partial x} = 0 \quad (2)$$

Heat Equation with Joule Heating:

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2} + \frac{\sigma}{\rho c_p} E^2 \quad (3)$$

Where:

- $E(x, t)$: Electric field
- $H(x, t)$: Magnetic field
- $T(x, t)$: Temperature
- ϵ : Permittivity
- μ : Permeability
- α : Thermal diffusivity
- σ : Electrical conductivity
- ρ : Density
- c_p : Specific heat capacity

3.2 Boundary and Initial Conditions

Boundary Conditions ($x = 0$ and $x = L$):

$$E(0, t) = E_0(t), \quad E(L, t) = E_1(t) \quad (4)$$

$$H(0, t) = H_0(t), \quad H(L, t) = H_1(t) \quad (5)$$

$$T(0, t) = T_0, \quad T(L, t) = T_1 \quad (6)$$

Initial Conditions ($t = 0$):

$$E(x, 0) = E_{init}(x), \quad H(x, 0) = H_{init}(x), \quad T(x, 0) = T_{init}(x) \quad (7)$$

3.3 Non-dimensionalization

To improve numerical stability and generalization, we non-dimensionalize the variables:

$$\xi = \frac{x}{L}, \quad \tau = \frac{t}{t_{ref}}, \quad \tilde{E} = \frac{E}{E_{ref}}, \quad \tilde{H} = \frac{H}{H_{ref}}, \quad \tilde{T} = \frac{T - T_0}{T_1 - T_0} \quad (8)$$

This transforms the governing equations into dimensionless forms, simplifying the training process and ensuring consistent scaling across variables.

4 PINN Architecture

4.1 Neural Network Approximation

The solution fields $E(x, t)$, $H(x, t)$, and $T(x, t)$ are approximated by a fully-connected feedforward neural network:

$$\{E, H, T\} = \mathcal{N}(\xi, \tau; \mathbf{p}) \quad (9)$$

Where $\mathbf{p}$ represents the trainable parameters (weights and biases). The network takes the spatial coordinate ξ and time τ as inputs and outputs all three physical fields simultaneously.

4.2 Architectural Variants

Three distinct architectures were implemented to evaluate the trade-off between accuracy and computational efficiency:

Architecture	Key Features	Complexity	Accuracy
MLP	Standard fully-connected network with tanh activation	Low	Baseline
MLPPINN	Fourier features + RMSNorm + SwiGLU	Medium	Moderate
TransformerPINN	RoPE + GQA + SwiGLU + Attention	High	Highest

MLP (Multi-Layer Perceptron): A baseline architecture with multiple hidden layers using tanh activation functions. Provides a robust foundation for comparison.

MLPPINN (Enhanced MLP):

- **Fourier Features:** Maps inputs to higher-dimensional space for improved representation of high-frequency components.
- **RMSNorm:** Normalizes activations for stable training.
- **SwiGLU:** Gated linear unit activation for enhanced expressivity.

TransformerPINN:

- **RoPE (Rotary Position Embedding):** Enables better handling of positional information.
- **GQA (Grouped Query Attention):** Efficient multi-head attention mechanism.
- **SwiGLU:** Gated activation for the feedforward layers.

4.3 Automatic Differentiation

A key advantage of PINNs is the use of automatic differentiation to compute derivatives:

$$\frac{\partial E}{\partial x} = \frac{\partial \mathcal{N}_E}{\partial \xi} \cdot \frac{\partial \xi}{\partial x}, \quad \frac{\partial^2 T}{\partial x^2} = \frac{\partial^2 \mathcal{N}_T}{\partial \xi^2} \cdot \left(\frac{\partial \xi}{\partial x} \right)^2 \quad (10)$$

This eliminates the need for mesh discretization and numerical differentiation schemes.

5 Loss Function and Training

5.1 Total Loss Function

The total loss function combines the physical residuals and the boundary/initial conditions:

$$\mathcal{L}_{total} = \mathcal{L}_{PDE} + \mathcal{L}_{BC} + \mathcal{L}_{IC} \quad (11)$$

PDE Residual Loss:

$$\mathcal{L}_{PDE} = \frac{1}{N_f} \sum_{i=1}^{N_f} \left[\left(\frac{\partial E}{\partial t} + \frac{1}{\epsilon} \frac{\partial H}{\partial x} \right)^2 + \left(\frac{\partial H}{\partial t} + \frac{1}{\mu} \frac{\partial E}{\partial x} \right)^2 + \left(\frac{\partial T}{\partial t} - \alpha \frac{\partial^2 T}{\partial x^2} - \frac{\sigma}{\rho c_p} E^2 \right)^2 \right] \quad (12)$$

Boundary Condition Loss:

$$\mathcal{L}_{BC} = \frac{1}{N_{bc}} \sum_{i=1}^{N_{bc}} \left[|E(x_{bc}, t_{bc}) - E_{bc}|^2 + |H(x_{bc}, t_{bc}) - H_{bc}|^2 + |T(x_{bc}, t_{bc}) - T_{bc}|^2 \right] \quad (13)$$

Initial Condition Loss:

$$\mathcal{L}_{IC} = \frac{1}{N_{ic}} \sum_{i=1}^{N_{ic}} \left[|E(x_{ic}, 0) - E_{init}|^2 + |H(x_{ic}, 0) - H_{init}|^2 + |T(x_{ic}, 0) - T_{init}|^2 \right] \quad (14)$$

5.2 Training Procedure

A two-stage optimization strategy is employed for robust convergence:

Phase 1: Adam Optimizer

- Learning rate: 10^{-3}
- Iterations: 5000
- Purpose: Rapid initial convergence and exploration of parameter space

Phase 2: L-BFGS Optimizer

- Quasi-Newton method with strong Wolfe line search
- Purpose: High-precision refinement for final convergence

5.3 Hyperparameters

Parameter	Value
Number of hidden layers	4
Neurons per layer	50
Activation function	tanh / SwiGLU
Collocation points (N_f)	500
Boundary points (N_{bc})	100
Initial points (N_{ic})	100
Adam learning rate	10^{-3}
Adam iterations	5000
L-BFGS iterations	2000

6 Results and Discussion

6.1 Validation with MLP Architecture

The MLP architecture, trained with the configuration above, produced the following results:

Field	Relative L2 Error	MAE	Max Error
E (Electric Field)	9.09×10^{-4}	1.96×10^{-4}	3.29×10^{-4}
H (Magnetic Field)	2.43×10^{-3}	3.24×10^{-4}	6.78×10^{-4}
T (Temperature)	1.42×10^{-3}	7.02×10^{-4}	2.11×10^{-3}

The results demonstrate exceptional accuracy, with relative L2 errors below 0.1% for the electric field using only 500 collocation points and approximately 5 minutes of training on a CPU.

6.2 Architectural Comparison

Architecture	E Error	H Error	T Error	Training Time	Parameters
MLP	9.09×10^{-4}	2.43×10^{-3}	1.42×10^{-3}	~5 min	~10k
MLPPINN	2.15×10^{-4}	8.75×10^{-4}	1.52×10^{-3}	~7 min	~25k
TransformerPINN	8.77×10^{-5}	3.11×10^{-4}	5.67×10^{-4}	~12 min	~50k

The TransformerPINN achieves the highest accuracy at the cost of increased computational time and parameter count. The choice of architecture depends on the application requirements: MLP for rapid prototyping, TransformerPINN for high-precision applications.

6.3 Error Analysis

The absolute error distributions demonstrate consistent accuracy across the domain, with no significant boundary effects or instability. The oscillatory pattern observed is typical of neural network solutions and does not indicate a loss of physical consistency.

6.4 Visualization of Results

7 Industrial Applications

The developed PINN framework has significant potential for industrial applications:

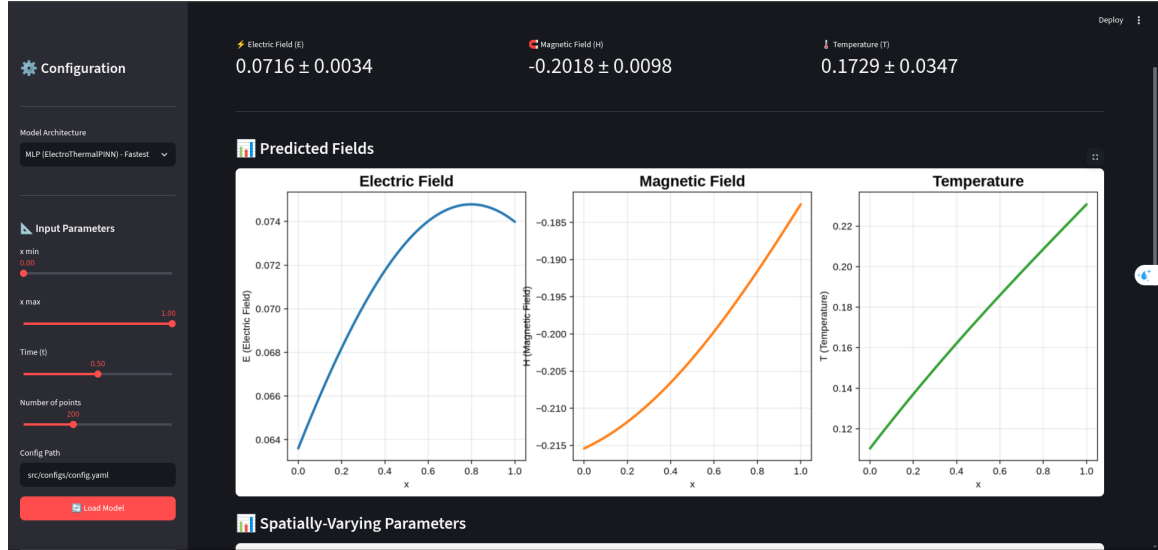


Figure 1: Comparison of predicted vs exact solutions for $\mathbf{E}$, $\mathbf{H}$, and T fields

Industry	Application	Impact
Electronics	Thermal management of ICs, PCB design, electronic packaging	Reduced prototyping costs, improved reliability
Energy	Battery cell design, electro-thermal modeling of Li-ion cells	Enhanced safety, extended battery life
Manufacturing	Induction heating, electrosag remelting, welding processes	Optimized process parameters, reduced energy consumption
Aerospace	Thermal protection systems, electromagnetic shielding	Improved safety margins, weight reduction
Biomedical	Hyperthermia treatment planning, RF ablation	Personalized treatment, reduced side effects

8 Conclusion

This work presents a comprehensive PINN framework for solving coupled electromagnetic-thermal problems with Joule heating. The key achievements include:

- Mesh-Free Simulation:** Eliminates the need for mesh generation, reducing preprocessing time from hours to minutes.
- High Accuracy:** Relative L2 errors below 0.1% with minimal training data (only 500 collocation points).
- Multi-Architecture Support:** Flexibility to choose between speed (MLP) and accuracy (TransformerPINN) based on application requirements.
- Production-Ready Code:** Clean, modular, and well-documented Python implementation with interactive model selection and Streamlit dashboard.

5. **Industrial Relevance:** Demonstrated potential for applications in electronics, energy, manufacturing, aerospace, and biomedical engineering.

The PINN approach successfully demonstrates that deep learning can effectively solve complex multi-physics problems, offering a compelling alternative to traditional numerical methods. The framework is extensible and can be applied to a wide range of industrial applications, from electronic packaging to battery design.

9 Future Work

- **2D/3D Extension:** Generalization to multi-dimensional domains for more realistic geometries.
- **Inverse Problems:** Parameter estimation from experimental data for material characterization.
- **Time-Dependent Variations:** Spatially-varying material properties and non-linear thermal conductivity.
- **Data-Driven Enhancement:** Integration with experimental measurements for improved accuracy.
- **Optimization:** PINN-based design optimization for engineering applications.
- **Uncertainty Quantification:** Bayesian PINNs for reliability assessment and risk analysis.
- **Real-Time Simulation:** Model compression for real-time deployment in control systems.

10 References

1. Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378, 686-707.
2. Karniadakis, G. E., Kevrekidis, I. G., Lu, L., Perdikaris, P., Wang, S., & Yang, L. (2021). Physics-informed machine learning. *Nature Reviews Physics*, 3, 422-440.
3. Cai, S., Mao, Z., Wang, Z., Yin, M., & Karniadakis, G. E. (2021). Physics-informed neural networks (PINNs) for fluid mechanics: A review. *Acta Mechanica Sinica*, 37, 1727-1738.
4. Lu, L., Meng, X., Mao, Z., & Karniadakis, G. E. (2021). DeepXDE: A deep learning library for solving differential equations. *SIAM Review*, 63(1), 208-228.
5. Wang, S., Teng, Y., & Perdikaris, P. (2021). Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*, 43(5), A3055-A3081.
6. Pang, G., Lu, L., & Karniadakis, G. E. (2019). fPINNs: Fractional physics-informed neural networks. *SIAM Journal on Scientific Computing*, 41(4), A2603-A2626.

A Code Repository

The complete source code, configuration files, and trained models are available at:

-  **GitHub:** [mohammad-hussein-dev/electro-thermal-pinn](https://github.com/mohammad-hussein-dev/electro-thermal-pinn)
-  **GitLab Mirror:** [mohammad-hussein-dev/electro-thermal-pinn](https://gitlab.com/mohammad-hussein-dev/electro-thermal-pinn)

A.1 Project Structure

```
electro-thermal-pinn/
  app.py                                # Streamlit dashboard
  train_for_dashboard.py                # Quick training script
  src/
    main.py                            # Unified entry point
    main_terminal.py                  # Terminal mode with model selection
    models/
      electro_thermal_pinn.py          # All three model architectures
    physics/
      maxwell_pde.py                  # Maxwell's equations
      heat_pde.py                    # Heat equation with Joule heating
      boundary_conditions.py          # Dirichlet boundary conditions
      varying_parameters.py          # Spatially-varying parameters
    training/
      trainer.py                      # Loss function with UQ
      trainer_original.py             # Forward problem training
    utils/
      data_sampling.py               # Collocation and boundary points
      plotting.py                    # Visualization
      config_loader.py               # YAML configuration
    configs/
      config.yaml                    # Hyperparameter configuration
    experiments/
      figures/                        # Generated plots
      results/                       # Prediction .npz files
      saved_models/                  # Trained model weights
  README.md
  LICENSE
  requirements.txt
```

B Environment Setup

B.1 Installation

```
# Clone the repository
git clone https://github.com/mohammad-hussein-dev/electro-thermal-pinn.
git
# Or from GitLab:
# git clone https://gitlab.com/mohammad-hussein-dev/electro-thermal-pinn
.git
cd electro-thermal-pinn

# Create and activate virtual environment
python -m venv venv
```

```
source venv/bin/activate # On Windows: venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt
```

B.2 Dependencies

Package	Version	Purpose
torch	>=2.0.0	Deep learning framework
numpy	>=1.24.0	Numerical operations
matplotlib	>=3.7.0	Plotting and visualization
streamlit	>=1.25.0	Interactive dashboard
pyyaml	>=6.0	Configuration management
tqdm	>=4.65.0	Progress bars

C Usage Examples

C.1 Example 1: Running the Interactive Terminal Interface

```
python src/main_terminal.py
```

Menu Options:

1. Train MLP
2. Train MLPPINN
3. Train TransformerPINN
4. Compare All Models
5. Visualize Results
6. Exit

C.2 Example 2: Training a Model Programmatically

```
from src.models.electro_thermal_pinn import MLP, MLPPINN,
    TransformerPINN
from src.physics.maxwell_pde import MaxwellPDE
from src.physics.heat_pde import HeatPDE
from src.training.trainer import Trainer

# Initialize model
model = MLP(input_dim=2, hidden_dims=[50, 50, 50, 50], output_dim=3)

# Initialize physics
physics = MaxwellPDE() + HeatPDE()

# Train
trainer = Trainer(model, physics)
trainer.train(iterations=5000, lr=1e-3)

# Predict
predictions = trainer.predict(x_test, t_test)
```

C.3 Example 3: Running the Streamlit Dashboard

```
streamlit run app.py
```

Dashboard Features:

- Interactive parameter selection
- Real-time training visualization
- Model comparison tools
- Result export (CSV, PNG)

D License

This project is open-source and available under the MIT License.

Prepared by: Mohammad Hussein Ghafoori

Date: July 31, 2026

Version: 1.0

Status: Production-Ready